

Customer Journey Analytics

A full-stack, multi-tenant platform that captures real user behaviour, visualises the conversion journey, and uses machine learning to predict drop-off and read customer sentiment.

React + Vite

Node.js / Express

PostgreSQL

FastAPI

XGBoost

BERT (Transformers)

Docker Compose

Feature Guide for the team presentation

Every capability explained: what it does, how it works under the hood, and why it matters.

Contents

1	System Overview & Architecture
2	Data Model
3	Tracking SDK & Event Ingestion
4	Multi-Tenancy & Site Isolation
5	Authentication, Roles & User Management
6	Conversion Funnel
7	Click & Movement Heatmap
8	Scroll-Depth Heatmap
9	Time-on-Page Analytics
10	Entry & Exit Pages
11	Rage-Click Detection
12	Navigation Paths
13	Conversion Influence
14	Engagement Scoring
15	Session Analytics (List & Detail)
16	Drop-off Risk Prediction (ML)
17	Risk Distribution
18	Sentiment Analysis & Insights
19	ML Training Pipeline
20	Deployment & Tech Stack

1 System Overview & Architecture

CJA is split into five cooperating services that you can run with a single `docker compose up`.

Service	Tech	Port	Responsibility
Dashboard (Frontend)	React + Vite	5173	The analyst-facing UI: charts, heatmaps, session explorer, admin screens.
Backend API	Node.js / Express	5000	Auth, tenant isolation, event ingestion, and all analytics computation.
ML Service	FastAPI (Python)	8000	Drop-off prediction (XGBoost) and sentiment analysis (BERT).
PostgreSQL	Postgres 16	5432	Stores sessions, events, features, users, sites, feedback.
Demo Stores	React + static HTML	3000 / 3001	Two sample shops that emit real tracking data into the platform.

How the pieces talk to each other

1. A visitor browses a demo store. The embedded **tracker.js** SDK captures clicks, scrolls, mouse moves, and page views.
2. Events are batched and POSTed to the backend `/api/track`, tagged with a `site_id`.
3. The backend writes raw events into PostgreSQL and incrementally maintains per-session aggregates.
4. An analyst logs into the dashboard. Each panel calls a scoped analytics endpoint that only returns data for sites the user is allowed to see.
5. For ML panels, the backend forwards features to the FastAPI service, which returns a drop-off probability or a sentiment label.

Design principles

- **Multi-tenant from the ground up** — every table, query, and API call is scoped by `site_id`.
- **Computation lives in the API layer** — most analytics are derived on-the-fly from raw events, so no nightly ETL is required for the demo.
- **Resilient client SDK** — the tracker wraps everything in try/catch and uses `sendBeacon` so it never breaks the host site.
- **ML as a separate service** — heavy Python dependencies are isolated, keeping the Node API lightweight.

2 Data Model

Six tables underpin the whole system. Note how `site_id` appears on every behavioural table to enforce tenant isolation.

Table	Key columns	Purpose
users	id, email, password (bcrypt), role	Dashboard accounts. Role is <code>admin</code> or <code>user</code> .
sites	site_id, display_name	Registry of tenants (e.g. <code>ecommerce_001</code> , <code>demo_bookstore_002</code>).
user_sites	user_id, site_id	Join table — which non-admin users may view which sites.
sessions	session_id, site_id, duration, total_clicks, max_scroll_depth, total_pages	One row per visit, with running aggregates maintained on each event.
events	session_id, event_type, x, y, page_url, scroll_depth, site_id, timestamp	The raw behavioural stream — clicks, scrolls, mouse moves, page views.
session_features	session_id, avg_scroll_depth, session_duration, total_clicks, pages_visited, max_funnel_stage	Pre-computed ML/scoring features per session.
feedback	site_id, message, sentiment_label, sentiment_score	Customer messages plus their analysed sentiment.

Aggregates are maintained live

When an event arrives, the ingestion handler upserts the session and then conditionally updates counters in the same request:

```
// click → total_clicks + 1
// scroll → max_scroll_depth = GREATEST(current, new)
// page_view → total_pages + 1
// always → end_time = NOW(), duration = end_time - start_time
```

Impact: dashboards stay fast because per-session metrics are already summarised — no expensive scans at read time for the common counters.

3 Tracking SDK & Event Ingestion

SDK `tracker.js` — the universal collector

What it does

A dependency-free JavaScript snippet you drop onto any website. It auto-starts on page load and silently records how visitors interact with the page.

How it works

Reads config from `window.CJA_CONFIG` (`site_id` , `track_endpoint` , `funnel`). It then:

- Creates a stable **session id** with `crypto.randomUUID()` , persisted in `sessionStorage` .
- Attaches passive listeners for **click**, **scroll**, **mousemove** and page views.
- **Throttles & debounces** noisy events (scroll every 1s, mouse moves debounced 200ms) to limit volume.
- Detects **funnel stages** by matching the path against the configured funnel (supports prefix matching like `/book/`).
- Tracks **SPA route changes** by patching `history.pushState` / `replaceState` and listening to `popstate` .

Reliability

Events are **batched** (max 10, or every 3s) and flushed on page exit using `navigator.sendBeacon` / `fetch keepalive` , so data isn't lost when the tab closes. Every handler is wrapped in try/catch and a no-op logger, so the tracker can never crash the host page.

Impact: a single `<script>` tag turns any site into a fully instrumented data source, without performance or stability risk.

API `POST /api/track` — ingestion endpoint

What it does

Receives one event or a batch of events from the SDK and persists them.

How it works

The `requireSiteId` middleware validates and normalises the `site_id` first. Each event is sanitised (floats rounded to integers for pixel columns), the session is upserted with `ON CONFLICT DO NOTHING` , the raw event is inserted, and the relevant session aggregate is bumped — all scoped to the tenant.

Impact: a single, robust write path keeps raw fidelity (for heatmaps) *and* rolled-up metrics (for speed) in sync.

4 Multi-Tenancy & Site Isolation

SECURITY Per-site data isolation

What it does

Guarantees that data from one website (tenant) can never leak into another's reports, and that each analyst only sees the sites they're authorised for.

How it works

- Every analytics route is guarded by `requireAuthorizedSiteId`, which (a) requires a `site_id` param, (b) lets a master admin see any site, and (c) for normal users checks the requested site against their `user_sites` rows — returning **403** if not permitted.
- Every SQL query filters by `site_id`, so isolation is enforced at the data layer, not just the UI.
- The `/api/sites` endpoint returns only the sites the caller may view, powering the dashboard's site switcher.

Impact: the same deployment can safely serve many clients/brands — a core requirement for a real SaaS analytics product.

5 Authentication, Roles & User Management

AUTH JWT login, register & first-run admin setup

What it does

Secures the dashboard with email/password accounts and signed tokens.

How it works

Passwords are hashed with **bcrypt**. On login a **JWT** (24h expiry) carrying `id`, `email`, and `role` is issued. The `authMiddleware` verifies the `Bearer` token on every protected route. A one-time `/setup-admin` route bootstraps the very first admin only when none exists.

RBAC Roles & site assignment

What it does

Separates **master admins** (see everything, manage users) from **business users** (see only assigned sites).

How it works

Admin-only endpoints (guarded by `adminMiddleware`) let an admin list users with their site access, promote/demote roles, and replace a user's assigned sites. Site reassignment runs in a DB **transaction** (delete + re-insert) and validates that every target site exists.

Impact: clean least-privilege access control — clients only ever see their own data, while your team retains full oversight.

6 Conversion Funnel

ANALYTICS GET /api/funnel

What it does

Shows how many visitors reach each stage of the purchase journey (e.g. Products → Cart → Checkout → Order Complete) and the step-by-step conversion rate.

How it works

Each site has a configured funnel (`SITE_FUNNELS`) supporting exact and prefix matches. The endpoint pulls all `page_view` events, finds the **furthest stage** each session reached, then counts sessions cumulatively per stage. Conversion rate for a stage = users at that stage ÷ users at the previous stage.

Impact: instantly pinpoints *where* the biggest leaks are — the single most actionable view for boosting revenue.

7 Click & Movement Heatmap

ANALYTICS **GET** `/api/heatmap?page=...`

What it does

Renders where users click and move their mouse on a specific page, overlaid as a heat cloud.

How it works

Returns the raw `x, y` coordinates of all `click` and `mousemove` events for the given page (and site). Page matching is trailing-slash tolerant so `/cart` and `/cart/` are treated as one. The frontend renders the density visualisation.

Impact: reveals what actually draws attention vs. what's ignored, guiding layout and call-to-action placement.

8 Scroll-Depth Heatmap

ANALYTICS [GET /api/scrollmap?page=...](#)

What it does

Shows how far down a page visitors actually scroll.

How it works

Collects the `scroll_depth` percentage from all `scroll` events for the page. The distribution reveals the point where most users stop reading.

Impact: tells you whether key content / CTAs sit above the "fold of attention" — critical for landing-page optimisation.

9 Time-on-Page Analytics

ANALYTICS GET /api/analytics/time-on-page

What it does

Reports how long visitors spend on each page — average, median, and 90th percentile.

How it works

Groups `page_view` events by session in time order, then computes the dwell time as the gap between consecutive page views. Outliers (negative gaps or > 1 hour) are discarded. It then derives avg / median / p90 per page and counts unique sessions.

Impact: distinguishes "engaging" pages from "confusing / stuck" pages; median + p90 avoid being skewed by idle tabs.

10 Entry & Exit Pages

ANALYTICS GET /api/analytics/entry-exit

What it does

Identifies where journeys most often **begin** and, more importantly, where they **end** (with an exit rate per page).

How it works

For each session, page URLs are **canonicalised** (query strings stripped, admin routes dropped, `/book/:id` & products collapsed) so noise doesn't skew results. The first canonical page is the entry, the last is the exit. A **statistical-noise floor** (≥ 3 sessions or 1% of traffic) prevents misleading "100% exit" off a single stray visit.

Impact: high-exit pages flag friction or dead-ends; entry pages reveal which content brings people in.

11 Rage-Click Detection

UX SIGNAL **GET /api/analytics/rage-clicks**

What it does

Surfaces spots where users repeatedly, frustratedly click — a strong signal of broken or unresponsive UI.

How it works

Within a session it scans clicks in time order and forms a **cluster** when ≥ 3 clicks land within **30px** and **600ms** of an anchor click. Nearby clusters (within 40px) are merged into "rage zones" and counted by how many sessions hit them.

Impact: directly exposes broken buttons / misleading non-clickable elements — quick wins that recover otherwise-lost conversions.

12 Navigation Paths

ANALYTICS GET /api/nav-paths

What it does

Lists the most common end-to-end routes users take through the site (e.g. `/products → /cart → /checkout`) and flags which paths converted.

How it works

Builds an ordered, de-duplicated list of canonical pages per session, joins them into a path string, and counts occurrences. It also computes overall conversion rate and the single most common path, returning the top N.

Impact: shows the "happy paths" that lead to purchase vs. the wandering paths that don't — informing navigation and merchandising.

13 Conversion Influence

ANALYTICS GET /api/conversion-influence

What it does

Compares behaviour of **converted** vs **dropped-off** sessions across clicks, scroll, duration, and pages visited — then auto-generates plain-English insights.

How it works

A session is "converted" if it visited an order/payment/success page (detected via `ILIKE`). It computes avg & median of each metric per group. Where converters out-perform droppers by $>1.5\times$, it emits an insight like *"Converted users scrolled 2.3x more than users who dropped off."*

Impact: translates raw numbers into shareable, decision-ready statements about what behaviours drive sales.

14 Engagement Scoring

SCORING GET /api/engagement-scores

What it does

Assigns every session an engagement score (0–100) and buckets sessions into **Highly** / **Moderately** / **Passive**.

How it works

A weighted blend of normalised signals:

```
score = avg_scroll_depth × 0.40  
       + normalized_duration × 0.35  
       + normalized_clicks   × 0.25
```

Duration and clicks are normalised against the site maximum. Thresholds: ≥ 70 highly, ≥ 40 moderately, else passive. It also reports average score, scores per funnel stage, and the top sessions.

Impact: a single, intuitive KPI to track content/UX quality over time and to segment audiences.

15 Session Analytics (List & Detail)

EXPLORER **GET** `/api/analytics/sessions & /sessions/:id`

What it does

A paginated, sortable table of individual sessions, drillable into a full event-by-event timeline.

How it works

The list supports server-side pagination (max 50/page) and **whitelisted** sort columns (duration, time, pages, funnel stage, scroll) to prevent SQL injection. It `LEFT JOIN`s `session_features` for richer metrics. The detail view returns session stats plus its first 300 ordered events.

Impact: lets analysts move from aggregate trends down to a single real user's journey — invaluable for debugging UX issues.

16 Drop-off Risk Prediction (Machine Learning)

ML **GET** `/api/predict/:session_id` → **FastAPI** `/predict`

What it does

Predicts the probability that a given session will **drop off** (fail to convert).

How it works

The backend gathers the session's features (duration, clicks, max scroll, pages) plus a derived **engagement score**, and POSTs them to the FastAPI service. There an **XGBoost classifier** returns a class and a probability. The model adapts to 4- or 5-feature input via `n_features_in_`. Errors from the ML service are surfaced cleanly to the caller.

Impact: moves from *describing* the past to *predicting* the future — enabling proactive interventions (offers, nudges) on at-risk sessions.

17 Risk Distribution

SCORING GET /api/analytics/risk-distribution

What it does

Buckets all sessions into **High** / **Medium** / **Low** drop-off risk for an at-a-glance health view.

How it works

A transparent, explainable scoring formula combines five normalised signals (duration 0.28, funnel stage 0.24, pages 0.20, scroll 0.18, clicks 0.10) into an engagement value; risk = $1 - \text{engagement}$, clamped to 0.02–0.98. Tiers: ≥ 0.67 high, ≥ 0.40 medium, else low. Returns the distribution plus the top 100 riskiest sessions.

Impact: a fast, model-independent risk pulse that complements the ML predictor and is easy to explain to stakeholders.

18 Sentiment Analysis & Insights

ML **POST** `/api/sentiment` → **FastAPI** `/sentiment`

What it does

Reads customer feedback text and classifies it as positive / neutral / negative with a confidence score.

How it works

The backend forwards the text to the FastAPI service, which runs a Hugging Face **BERT** `sentiment-analysis` pipeline and returns a label and score.

ANALYTICS **GET** `/api/analytics/sentiment-insights`

What it does

Aggregates stored feedback into counts, percentages, and representative sample quotes per sentiment.

How it works

Queries the `feedback` table grouped by `sentiment_label`, computes percentages, and pulls the 3 strongest positive/neutral and 3 most negative messages as illustrative snippets.

Impact: turns unstructured customer voice into a quantified signal you can trend and act on, with real quotes for context.

19 ML Training Pipeline

PIPELINE `train_model.py`

What it does

Trains and saves the XGBoost drop-off model directly from live session data.

How it works

Connects to Postgres, labels each session (reached payment/order-confirmed $\Rightarrow$ converted, else drop-off), and trains an `XGBClassifier` with a stratified train/test split. It handles class imbalance via **balanced sample weights** and reports accuracy, **balanced accuracy**, a majority-class baseline, classification report, and confusion matrix before saving `xgboost_model.pkl`.

Note: guard-rails raise clear errors if there's only one class or too few minority samples — so training fails loudly rather than producing a useless model.

Impact: a reproducible, data-driven retraining loop — the model improves as more real traffic accumulates.

20 Deployment & Tech Stack

The entire platform is containerised. A one-time setup profile creates tables and seeds the admin; then a single command brings up all services.

```
# first time only – create tables & seed admin
docker compose --profile setup up db-setup

# start everything
docker compose up --build
```

Layer	Technology	Why
Frontend	React 18 + Vite	Fast dev experience, component-based dashboards.
API	Node.js + Express	Lightweight, ideal for I/O-bound analytics queries.
Auth	JWT + bcrypt	Stateless, standard, secure.
Database	PostgreSQL 16	Reliable relational store with rich SQL for analytics.
ML	FastAPI, XGBoost, Transformers (BERT)	Python ecosystem for modelling, isolated from the API.
Orchestration	Docker Compose	One-command, reproducible multi-service environment.

Talking points for the presentation

- **End-to-end story:** raw clicks → aggregated metrics → predictive ML, all in one platform.
- **Production-minded:** multi-tenant isolation, RBAC, batched/resilient tracking, injection-safe queries.
- **Explainable + predictive:** transparent scoring formulas sit alongside ML models, so insights are trustworthy *and* forward-looking.
- **Extensible:** new sites need only a row in `sites` and the tracker snippet; new analytics are just new scoped endpoints.